ALCOR Timing Calibration Report

1 Scope and Objective

This document summarizes the timing model, calibration chain, validation logic, and main interpretation caveats for the ALCOR timing analysis. It is intended as the technical note for the calibration procedure and for reading the produced outputs.

The goal of the calibration chain is to improve timing resolution by correcting:

- fine-time non-linearity,
- time-walk through a ToT-dependent correction,
- residual channel-to-channel offsets.

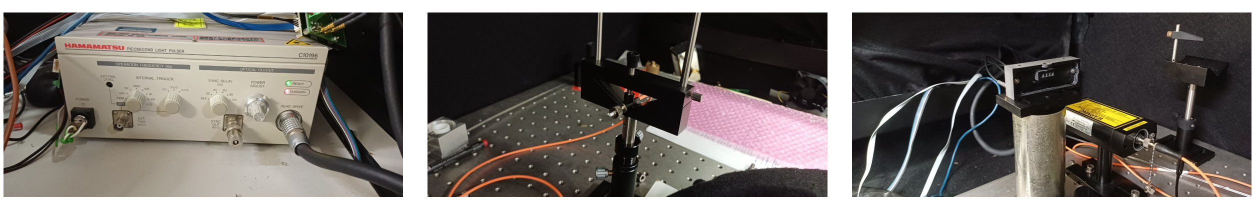
The target is sub-nanosecond timing, with an aspirational scale of a few hundred picoseconds if the detector response and the dataset quality support it.

2 Experimental Setup

2.1 Calibration setup

The calibration uses a Hamamatsu picosecond light pulser (PLP-10 system) controlled by the C10196 controller. The PLP-10 provides laser pulses with typical width of about 70 ps full width at half maximum (FWHM) and repetition rates up to 100 MHz, with external trigger capability up to about 50 MHz.[1] During calibration it was operated at 100 kHz.

The light pulser illuminates two SiPMs from the onsemi Micro-J series (4×4 mm^2 package, active area 3.93×3.93 mm^2). The sensors provide a fast output obtained by capacitive coupling, used here for timing.[2]



Laser source **Emitter** **SiPM sensors**

Figure 1: Calibration setup photographs.

[1] Hamamatsu PLP-10 datasheet, `https://www.hamamatsu.com/content/dam/hamamatsu-photonics/sites/documents/99_SALES_LIBRARY/sys/SOCS0003E_PLP-10.pdf`.

[2] onsemi Micro-J series datasheet, `https://www.onsemi.com/download/data-sheet/pdf/microj-series-d.pdf`.

2.2 Scintillating fiber setup

The second setup uses a 1 m scintillating fiber, with both ends coupled to the same two SiPMs. A ^{90}Sr beta source (10 kBq) is placed at the midpoint of the fiber to generate signals.

3 Dataset Roles and Analysis Flow

Two dataset roles are assumed:

- **Calibration run**: used to build the fine and channel calibration constants.
- **Golden run**: used as an independent validation sample.

Recommended rule:

- build the calibrations on the calibration run,
- validate them on a distinct golden run.

Using the same dataset both to build and validate the calibration can hide overfitting or dataset-specific biases.

The intended flow is:

1. decode raw data into ROOT files,
2. build the fine-time calibration,
3. build the channel-level ToT and offset calibration,
4. run coincidence analysis with and without calibration,
5. compare calibration-run and golden-run behavior.

The analysis works on decoded ROOT files under `data/<run>/kc705-196/decoded/alcdaq.fifo_*.root`.

4 Timing Representation

The raw timestamp is represented as

$$\text{time\_tick} = \text{rollover} \cdot 32768 + \text{coarse},$$
$$\text{tick\_ns} = \frac{1000}{\text{clock\_mhz}}.$$

When fine correction is enabled, the calibrated time is

$$\text{time\_ns} = (\text{time\_tick} - \text{fine\_fraction})\,\text{tick\_ns}.$$

When fine correction is disabled, the code uses

$$\text{time\_ns} = \text{time\_tick}\,\text{tick\_ns}.$$

4.1 Global TDC index

Fine calibration is performed per physical TDC, not only per local `tdc=0..3`. The code uses the global TDC index

$$\text{tdc\_index} = \text{tdc} + 4 \cdot \text{pixel} + 16 \cdot \text{column} + 128 \cdot \text{fifo}.$$

This matters because:

- each FIFO contains 128 physical TDCs,
- different columns and pixels must not be merged into the same calibration entry,
- any change in the TDC indexing convention requires the fine and channel calibrations to be regenerated.

5 Fine-Time Calibration

5.1 Motivation

The raw fine counter is not linearly proportional to time. The measured fine-code distribution reflects the phase relation between an asynchronous discriminator crossing and the TDC clock, and the conversion can be distorted across the 512 fine bins.

5.2 CDF LUT method

For each physical TDC:

- build a 512-bin histogram of `fine_raw`,
- compute the cumulative distribution,
- convert the cumulative distribution into a fine-time lookup table.

For bin b with count c_b and total entries N:

$$\begin{aligned}
\text{cum}_b &= \sum_{k \le b} c_k, \\
\text{frac}_b &= \text{clamp}\left(\frac{\text{cum}_b - 0.5\, c_b}{N}, 0, 1\right), \\
\text{fine_fraction} &= \text{frac}_b - 0.5.
\end{aligned}$$

The LUT is stored in `hFineLut` with:

- X: global TDC index,
- Y: raw fine code,
- Z: calibrated fine fraction.

5.3 Min/max and linear fallback

The same calibration step also extracts `min` and `max` values from configurable quantiles. If the LUT is disabled or unavailable, the code falls back to a linear mapping:

$$\text{fine\_fraction} = \frac{\text{fine\_raw} - \text{min}}{\text{max} - \text{min}},$$
$$\text{if fine\_raw} > \text{cut, then fine\_fraction} -= 1,$$

with

$$\text{cut} = \frac{\text{min} + \text{max}}{2}.$$

5.4 When the LUT assumption is valid

The CDF LUT approach is appropriate when the arrival phase relative to the clock is effectively random. Typical good cases are:

- dark counts,
- asynchronous detector signals,
- calibration pulses not phase-locked to the TDC clock.

In that case the fine histogram should look roughly like a plateau with edge roll-off. If the fine code instead collapses into narrow peaks, the LUT can become biased and a different calibration strategy may be needed.

5.5 Uniformity diagnostics

Differential non-linearity (DNL) quantifies the bin-to-bin deviation from a uniform population:

$$\text{DNL}(b) = \frac{c_b}{\langle c \rangle} - 1.$$

Integral non-linearity (INL) is the cumulative deviation from the ideal CDF:

$$\text{INL}(b) = F_{\text{emp}}(b) - \frac{b - 0.5}{B}.$$

Figures 2 and 3 show typical examples used to check that the LUT improves the fine-to-time mapping.

6 Channel Calibration

The channel calibration has two parts:

- a ToT-dependent correction,
- a residual static offset per channel.

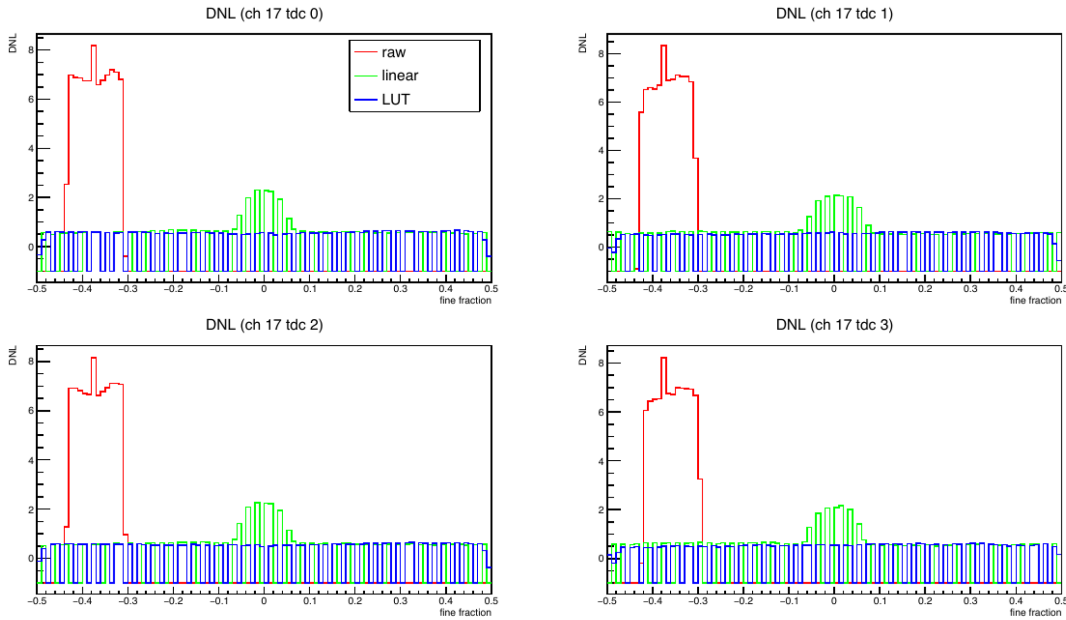


Figure 2: DNL example for one channel. The LUT-corrected curve is closer to the ideal uniform response than the raw and simple linear cases.

6.1 Time-walk correction

Leading-edge timing depends on pulse amplitude. Time-over-Threshold (ToT) is used as a proxy for amplitude, so the calibration builds a correction as a function of ToT.

Procedure:

1. choose a reference channel,
2. build Δt versus ToT for each channel,
3. estimate the correction using the median in each ToT bin,
4. apply the correction once and recompute,
5. use the refined curve as the final channel correction.

Important details:

- the correction is applied only to leading edges,
- a valid trailing edge must exist so that ToT is defined,
- if `max_duration <= 0`, the ToT correction is skipped.

Application convention:

$$t_{\text{lead}} -= \text{CorrectionNs}(\text{channel}, \text{ToT}).$$

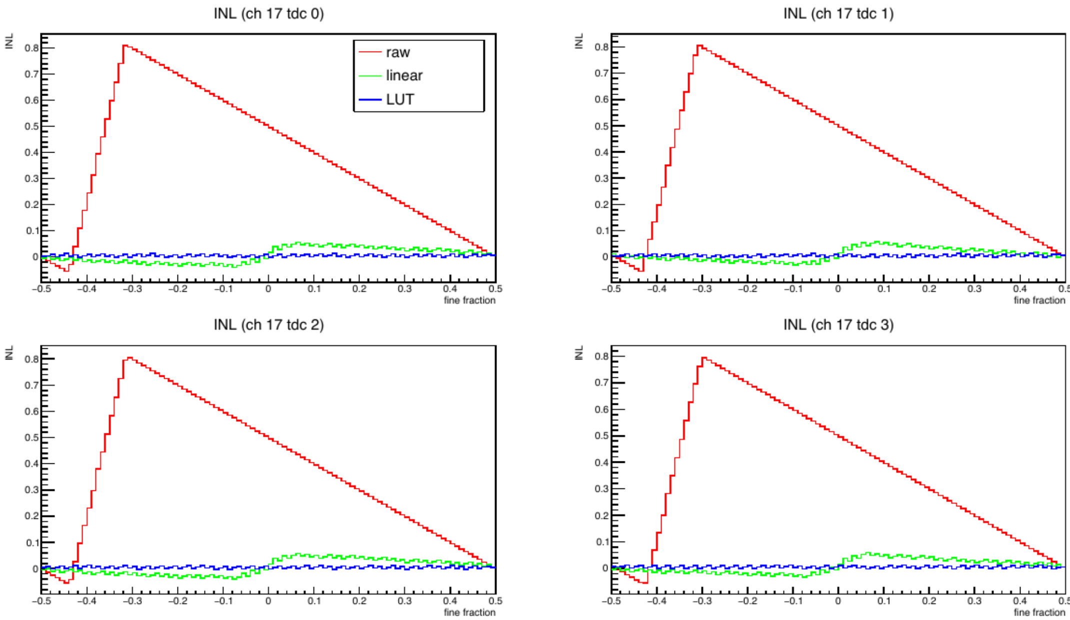


Figure 3: INL example for one channel. The LUT reduces the cumulative non-linearity compared to the raw and simple linear cases.

6.2 Channel offsets

After the ToT-dependent part is applied, a residual fixed offset may remain. The channel calibration stores a static offset for each channel to absorb this remaining shift.

The current ROOT output stores both:

- `hChanCalib_chN`: ToT-dependent correction histogram,
- `hChanOffset`: global per-channel offset summary.

6.3 Symmetric reference mode

In symmetric mode, the correction is split between the calibrated channel and the reference channel, rather than keeping the reference artificially fixed.

This is often useful for two-channel datasets, but it should be interpreted carefully once more channels contribute to the same reference scale.

7 Coincidence Analysis

7.1 Configuration and outputs

Coincidence studies are configured through `config/*.txt`. Supported line formats are:

- pair line: `chA chB [window_ns]`,
- group line: `group ch1 ch2 ch3 [window=ns]`.

Typical outputs include:

- search histograms,
- coincidence histograms,
- FWHM estimates,
- Δt versus fine 2D plots,
- ROOT and TXT summaries.

For a simple pair configuration, the main coincidence summary in the TXT output is usually the line that reports `fwhm_bkg_sub=...`.

7.2 Important implementation caveat

The current `coincidence_rdf.cxx` implementation does not enforce unique or nearest-neighbour matching for pair coincidences. Instead, it fills all hit pairs found inside the coincidence window.

Implications:

- coincidence counts can be inflated when multiple hits are present in the same window,
- the width of the coincidence peak can be biased by combinatorial matches,
- dataset-to-dataset comparisons must be interpreted with this in mind.

This behavior is especially important when pair multiplicity is not negligible.

7.3 Fine cut

An optional fine cut rejects hits near the wrap point:

$$|\text{fine\_raw} - \text{cut}| \leq \text{fine\_cut},$$

where `cut` is derived from the calibrated `min` and `max` for the corresponding TDC.

8 Validation Metrics for Fine Calibration

`fine_validation_rdf` reports several metrics to assess how well the calibrated fine fraction approaches a uniform distribution on $[-0.5, 0.5]$.

Main metrics:

- KS D: Kolmogorov-Smirnov distance to uniform,
- KS p-value: larger is better,
- AD (A^2): Anderson-Darling statistic,
- mean: should be close to 0,
- std: should be close to 0.288675 for a uniform distribution on $[-0.5, 0.5]$.

Reported categories:

- `raw`: no LUT correction,
- `intrinsic`: LUT evaluated on the same sample used to build it,
- `cross`: LUT evaluated on a held-out split.

Typical supporting plots include CDF residuals, DNL, and INL.

9 Reading the Output Files

9.1 Fine calibration output

`fine_calibration.root` typically contains:

- `hFineMin`,
- `hFineMax`,
- `hFineEntries`,
- `hFineLut`,
- the `fine_calib` tree.

Useful checks:

- number of valid TDCs,
- `min` and `max` values for the active TDCs,
- whether the active TDC set matches the channels actually used in the run.

9.2 Channel calibration output

`channel_calibration.root` typically contains:

- `hChanOffset`,
- `hChanCalib_ch0..31`,
- metadata such as reference channel, window, ToT range, and symmetric-reference mode.

Useful checks:

- per-channel offset magnitudes,
- number of nonzero ToT bins per active channel,
- whether most of the correction is a static offset or a real ToT-dependent shape.

9.3 Coincidence output

The coincidence TXT output is the fastest place to compare runs. For a simple pair analysis, check:

- pair definition,
- coincidence count,
- `fwhm_bkg_sub`,
- background estimate,
- left and right half-maximum crossings.

The PDF is then used to inspect whether the peak shape and Δt versus fine behavior are physically sensible.

10 Practical Validation Strategy

Recommended comparison sequence:

1. build fine and channel calibration on the calibration run,
2. compare the calibration run without calibration versus with calibration,
3. compare the golden run without calibration versus with calibration,
4. inspect Δt versus fine and, if relevant, Δt versus ToT,
5. confirm that the calibrated result improves, or at least does not degrade, the independent golden run.

If the calibration run improves but the golden run worsens, likely explanations include:

- overfitting to the calibration sample,
- unstable ToT or offset extraction,
- coincidence-window combinatorics,
- a mismatch between the calibration sample and the golden sample.

10.1 Example comparison plot

Figure 4 shows an example visual comparison from the calibration sample. Such plots are illustrative and should always be interpreted together with the current numeric outputs for the active run identifiers.

No calibration **With calibration**

Figure 4: Example calibration-run comparison using the coincidence summary page.

11 Documentation Policy

Performance statements should be tied to specific generated outputs and run identifiers, not kept as timeless claims in this document.

In particular:

- do not assume a fixed ToT-correction amplitude across datasets,
- do not assume a fixed resolution improvement,
- always quote the relevant output artifact or regenerate the result on the current calibration and golden runs.